

Code Review Report

Language: python | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 17:26 UTC

Overall Assessment

This pull request contains 8 findings with a risk score of 30, indicating a high level of risk. The majority of the findings (6) are critical, with 2 being high severity.

Merge readiness: Not ready for merge due to high-risk findings

Code health: 0/100

Findings Summary

Total findings: 8

Risk score: 30

Severity breakdown: Low 0, Medium 0, High 2, Critical 6

Type	Severity	Line	Message	Recommendation
Hardcoded Password	High	1	password = "admin123"	Store passwords in environment variables.
Hardcoded API Key	High	3	api_key = "123456789"	Store secrets securely.
SQL Injection	Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
Dangerous Dynamic Execution	Critical	9	eval(input())	Avoid dynamic code execution.
Dangerous Dynamic Execution	Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
Command Injection	Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.
SQL Injection	Critical	7	Unsafe SQL string concatenation.	Use parameterized queries.
Dangerous Dynamic Execution	Critical	9	eval(input()) usage.	Remove eval.

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
SQL Injection	Critical	High risk of data breach	Use parameterized queries to prevent SQL injection
Dangerous Dynamic Execution	Critical	High risk of code injection	Replace eval() and exec() with safer alternatives

Finding	Severity	Impact	Suggested Action
Command Injection	Critical	High risk of executing malicious commands	Use subprocess.run() with shell=False
SQL Injection	Critical	High risk of data breach	Use parameterized queries to prevent SQL injection
Dangerous Dynamic Execution	Critical	High risk of code injection	Replace eval() and exec() with safer alternatives
Dangerous Dynamic Execution	Critical	High risk of code injection	Replace eval() and exec() with safer alternatives
Hardcoded Password	High	High risk of password exposure	Store passwords in environment variables
Hardcoded API Key	High	High risk of API key exposure	Store API keys securely using environment variables or secure secret storage

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess

subprocess.run("dir", shell=True)
```